

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Universidad Técnica Federico Santa María

Departamento de Industrias

Campus Vitacura – Santiago, Chile

Sistemas de Información para la Gestión (ICN-292)

Semestre 2026/02

Entregable SQL (1/2)

Laboratorio 4: Base de Datos Austral Editores

Estudiante:	[Nombre Completo del Estudiante]
Rol USM:	[Rol USM]
Profesores:	José Miguel González
	Patricio Cifuentes
	José Luis Sáez Tamayo
Fecha de Entrega:	7 de Noviembre de 2026
Repositorio GitHub:	https://github.com/hctrsa04/ICN292-Lab4-Sanchez-Hector.git

Entregable 1: Creación de la Base de Datos y Modelo Entidad–Relación

1.1 Reconstrucción de la Base de Datos `austral`

De acuerdo con el escenario planteado para **Austral Editores**, se construyó el script SQL completo que inicializa la base de datos desde un estado limpio e incorpora tanto las tablas base del catálogo inicial (`autores` y `libros`) como las tablas adicionales requeridas (`colecciones` y `pagos`).

```sql
DROP DATABASE IF EXISTS austral;
CREATE DATABASE austral;
USE austral;

-- Tabla 1: autores
CREATE TABLE autores (
    id_autor INT AUTO_INCREMENT,
    nombre VARCHAR(80) NOT NULL,
    pais VARCHAR(60),
    genero ENUM('Cómic', 'Ensayo', 'Infantil', 'Novela', 'Poesía') NOT
       NULL,
    PRIMARY KEY (id_autor)
);

-- Tabla 2: colecciones
CREATE TABLE colecciones (
    id_coleccion INT AUTO_INCREMENT,
    titulo VARCHAR(120) NOT NULL,
    fecha_publicacion DATE NOT NULL,
    PRIMARY KEY (id_coleccion)
);

-- Tabla 3: libros
CREATE TABLE libros (
    id_libro INT AUTO_INCREMENT,
    id_autor INT,
    id_coleccion INT,
    titulo VARCHAR(120) NOT NULL,
    paginas INT,
    ejemplares_vendidos INT,
    anio INT,
    PRIMARY KEY (id_libro),
    FOREIGN KEY (id_autor) REFERENCES autores(id_autor),
    FOREIGN KEY (id_coleccion) REFERENCES colecciones(id_coleccion)
);

-- Tabla 4: pagos
CREATE TABLE pagos (
    id_pago INT AUTO_INCREMENT,
    id_autor INT,
    fecha_pago DATE NOT NULL,
    monto DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id_pago),
    FOREIGN KEY (id_autor) REFERENCES autores(id_autor)
);
```

1.2 Modelo Entidad–Relación Reconstruido

A través de la función de ingeniería inversa de MySQL Workbench (*Database >Reverse Engineer...*), se generó el diagrama Entidad–Relación completo con la vinculación mediante claves foráneas (`FOREIGN KEY`) entre las entidades `autores`, `libros`, `colecciones` y `pagos`.

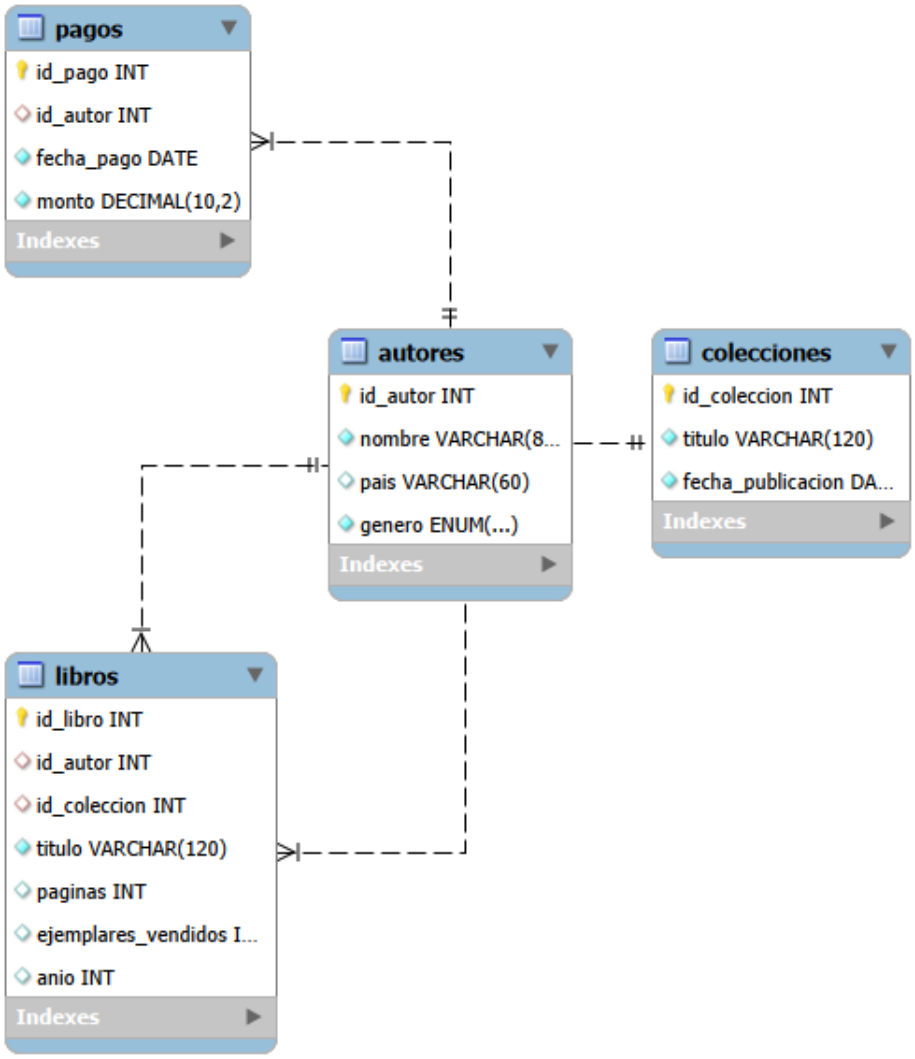


Figura 1: Modelo Entidad–Relación de la base de datos austral reconstruido en MySQL Workbench.

Entregable 2: Consultas SQL Desarrolladas y Evidencia de Ejecución

Consulta 1

Requerimiento: Muestre el nombre y el género de los autores cuyo país no es Chile, incluyendo a quien no tiene país registrado, ordenados por nombre. En el informe explique, en una línea, qué devuelve la comparación `pais <>'Chile'` y por qué.

```
SELECT nombre, genero
FROM autores
WHERE pais <> 'Chile' OR pais IS NULL
ORDER BY nombre ASC;
```

Explicación requerida: La comparación `pais <>'Chile'` descarta por defecto los valores `NULL` (como el de Kenji Arakawa) porque en SQL ningún valor vacío se evalúa como verdadero ni falso frente a operadores relacionales, haciendo obligatorio incluir explícitamente `OR pais IS NULL`.

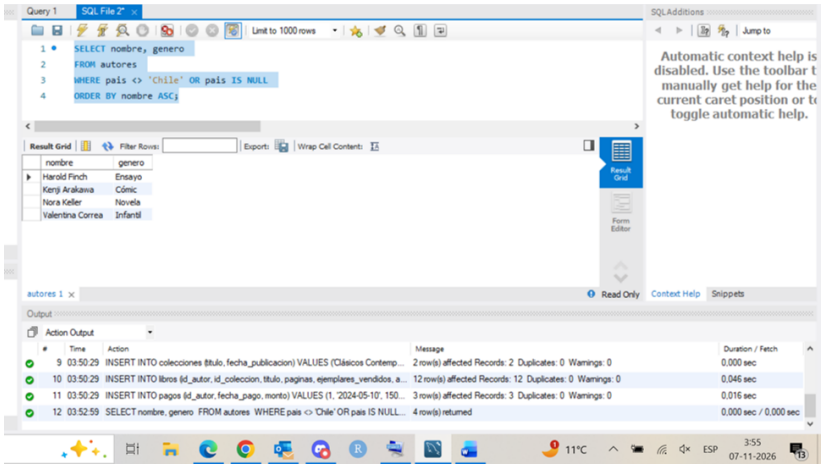


Figura 2: Resultado de la Consulta 1 en MySQL Workbench mostrando autores cuyo país difiere de Chile o es nulo.

Consulta 2

Requerimiento: Liste el título y las páginas de los libros que tienen entre 190 y 220 páginas, incluidos los dos valores, de menor a mayor número de páginas.

```sql
SELECT titulo, paginas
FROM libros
WHERE paginas BETWEEN 190 AND 220
ORDER BY paginas ASC;
```

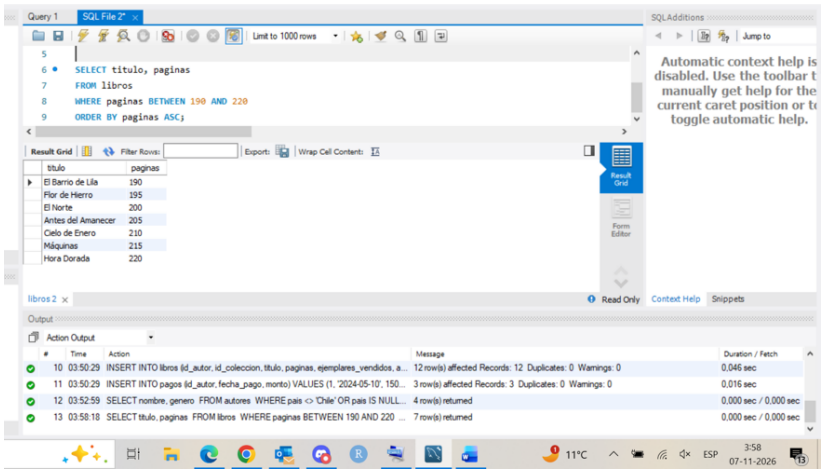


Figura 3: Resultado de la Consulta 2 listando libros con páginas en el rango [190, 220].

Consulta 3

Requerimiento: Entregue una sola fila con el número de libros del catálogo, la suma de ejemplares vendidos, el promedio de páginas, el año más antiguo y la mayor cantidad de ejemplares vendidos de un libro. Cada columna lleva un alias.

```sql
SELECT
    COUNT(*) AS total_libros,
    SUM(ejemplares_vendidos) AS total_vendidos,
    AVG(paginas) AS promedio_paginas,
    MIN(anio) AS anio_mas_antiguo,
    MAX(ejemplares_vendidos) AS max_vendidos
FROM libros;
```

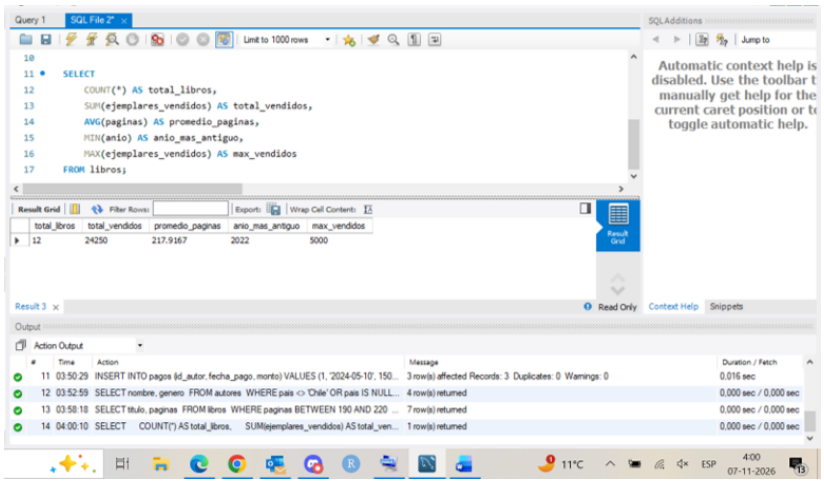


Figura 4: Resultado de la Consulta 3 con funciones de agregación sobre la tabla libros.

Consulta 4

Requerimiento: Liste los géneros distintos de los autores, una vez cada uno, en orden alfabético.

```sql
SELECT DISTINCT genero
FROM autores
ORDER BY genero ASC;
```

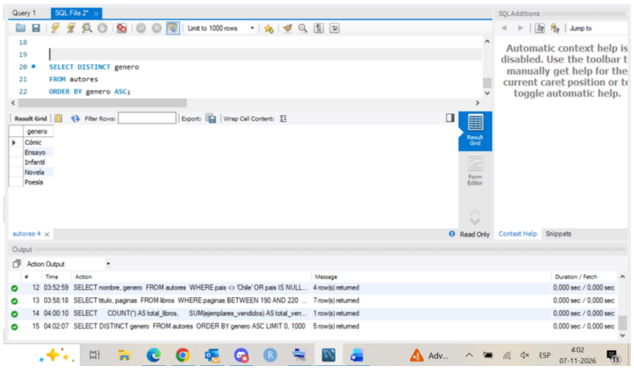


Figura 5: Resultado de la Consulta 4 mostrando la lista de géneros sin duplicados.

Consulta 5

Requerimiento: Muestre el nombre de cada autor que tiene más de un libro publicado en 2024 y cuántos son. Ordene por nombre.

```sql
SELECT
    a.nombre,
```

```
    COUNT(*) AS libros_2024
FROM autores a
JOIN libros l ON a.id_autor = l.id_autor
WHERE l.anio = 2024
GROUP BY a.id_autor, a.nombre
HAVING COUNT(*) > 1
ORDER BY a.nombre ASC;
```

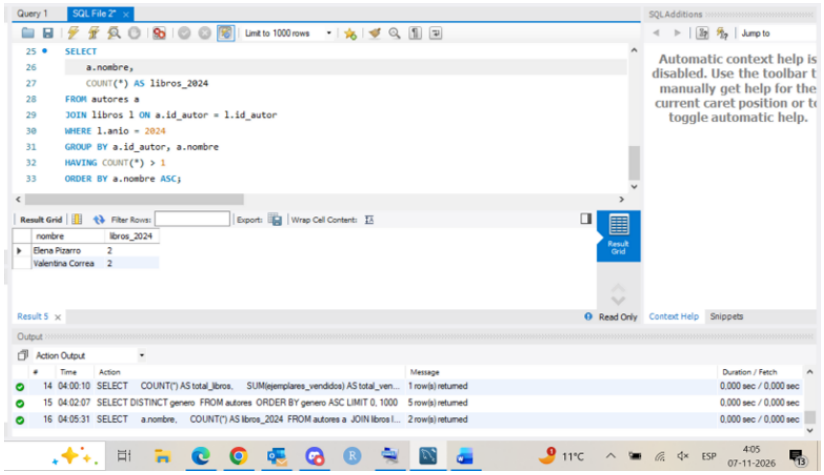


Figura 6: Resultado de la Consulta 5 filtrando autores agrupados con más de una publicación en 2024.

Consulta 6

Requerimiento: Liste el título y los ejemplares vendidos de los libros de 2024 que superan los 2000 ejemplares, de mayor a menor. Muestre solo los tres primeros.

```
SELECT titulo, ejemplares_vendidos
FROM libros
WHERE anio = 2024 AND ejemplares_vendidos > 2000
ORDER BY ejemplares_vendidos DESC
LIMIT 3;
```

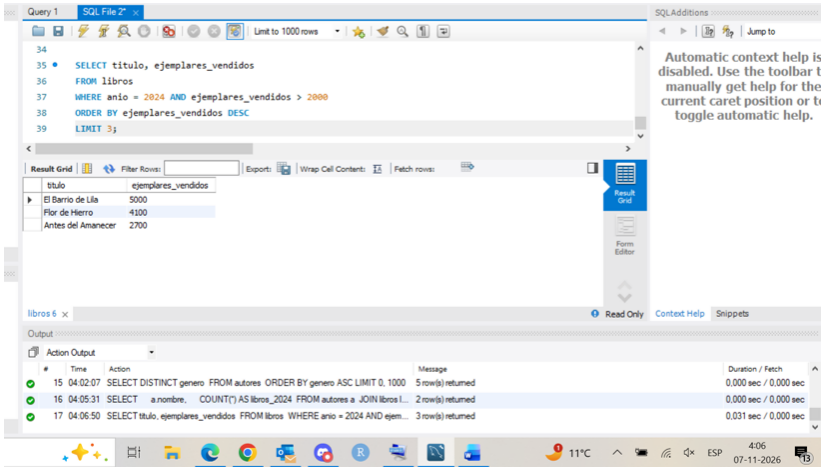


Figura 7: Resultado de la Consulta 6 listando los 3 libros más vendidos del año 2024.

Consulta 7

Requerimiento: Inserte el libro `'Papel Mojado'`, de Rodrigo Bahamonde, con 260 páginas, 50 ejemplares vendidos y año 2024. Después elimínelo con un `DELETE` que filtre por título. Escriba un `SELECT` antes y otro después.

```sql
-- 1. Insercion
SET SQL_SAFE_UPDATES = 0;
INSERT INTO libros (id_autor, id_coleccion, titulo, paginas,
    ejemplares_vendidos, anio)
VALUES (3, 1, 'Papel Mojado', 260, 50, 2024);

-- 2. SELECT previo
SELECT * FROM libros WHERE titulo = 'Papel Mojado';

-- 3. DELETE
DELETE FROM libros WHERE titulo = 'Papel Mojado';
SET SQL_SAFE_UPDATES = 1;

-- 4. SELECT posterior
SELECT * FROM libros WHERE titulo = 'Papel Mojado';
```

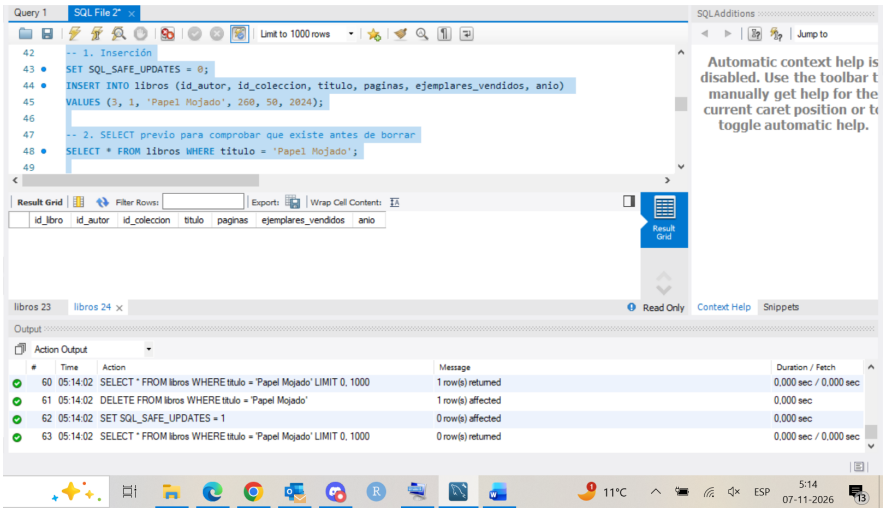


Figura 8: Resultado de la Consulta 7 evidenciando la inserción previa y posterior eliminación del registro.

Consulta 8

Requerimiento: Sume 100 ejemplares vendidos a `'Cielo de Enero'`. El `UPDATE` debe calcular el valor nuevo a partir del actual. Entregue tres capturas: el `SELECT` previo, el `UPDATE` y el `SELECT` posterior.

```sql
-- 1. SELECT previo
SELECT titulo, ejemplares_vendidos
FROM libros
WHERE titulo = 'Cielo de Enero';

-- 2. UPDATE
SET SQL_SAFE_UPDATES = 0;
UPDATE libros
SET ejemplares_vendidos = ejemplares_vendidos + 100
WHERE titulo = 'Cielo de Enero';
SET SQL_SAFE_UPDATES = 1;
```

```sql
-- 3. SELECT posterior
SELECT titulo, ejemplares_vendidos
FROM libros
WHERE titulo = 'Cielo de Enero';
```

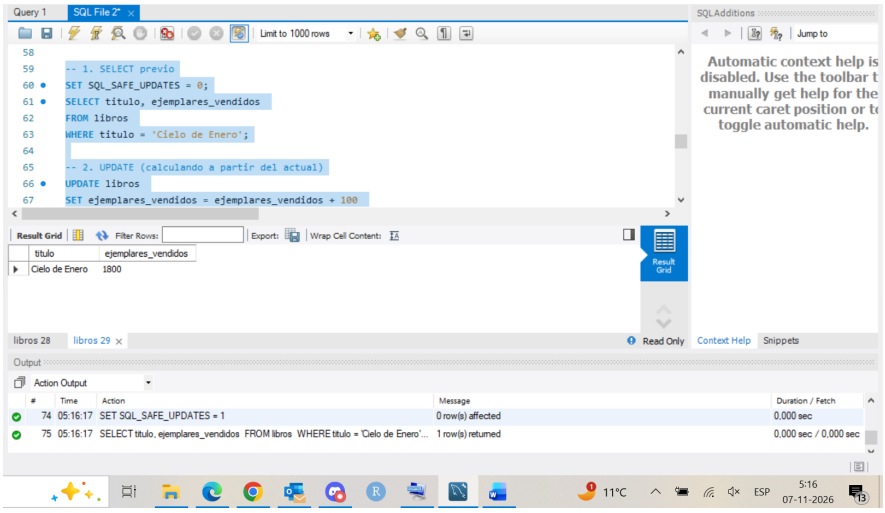


Figura 9: Resultado de la Consulta 8 confirmando la actualización calculada sobre los ejemplares vendidos.

Consulta 9

Requerimiento: Liste el título de cada libro, el título de la colección a la que pertenece y la fecha de publicación de la colección.

```sql
SELECT
    l.titulo AS libro,
    c.titulo AS coleccion,
    c.fecha_publicacion
FROM libros l
JOIN colecciones c ON l.id_coleccion = c.id_coleccion;
```

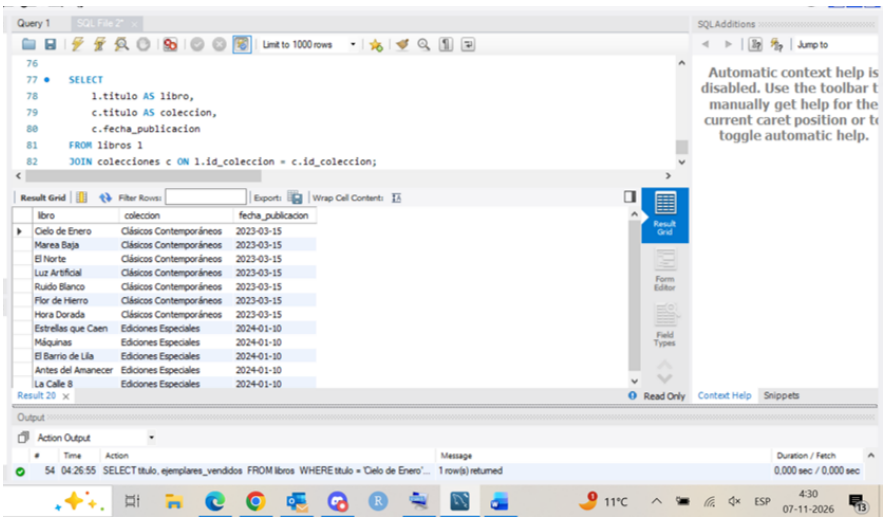


Figura 10: Resultado de la Consulta 9 mostrando el cruce (JOIN) entre libros y colecciones.

Consulta 10

Requerimiento: Liste los pagos hechos a los autores con fecha y monto, y luego muestre el total pagado a cada autor.

```sql
-- Sentencia 1: Detalle de pagos
SELECT
    a.nombre,
    p.fecha_pago,
    p.monto
FROM pagos p
JOIN autores a ON p.id_autor = a.id_autor;

-- Sentencia 2: Total pagado por autor
SELECT
    a.nombre,
    SUM(p.monto) AS total_pagado
FROM pagos p
JOIN autores a ON p.id_autor = a.id_autor
GROUP BY a.id_autor, a.nombre;
```

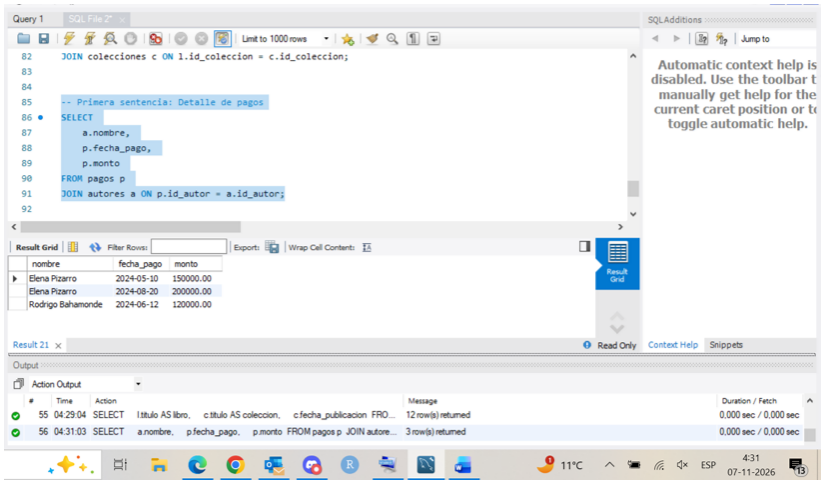


Figura 11: Detalle de los pagos individuales registrados por autor (Sentencia 1).

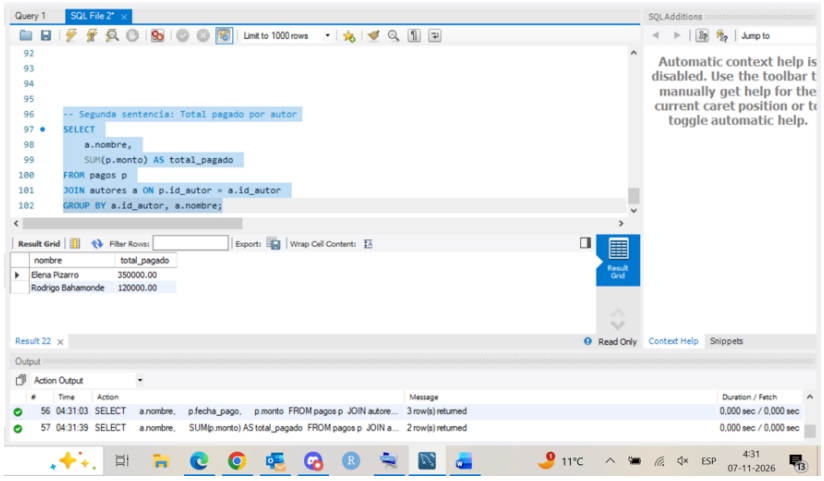


Figura 12: Monto total acumulado pagado a cada autor (Sentencia 2).